

Curso Fundamento VISÃO

Do zero absoluto até a realidade do projeto — sem economia de didática

Preparado por ÍRIS · Agosto de 2026 · Projeto VISÃO / Juan Guerra

Este material foi pedido **extremamente detalhado**. Ele começa onde você está agora — sem pressupor quase nada — e sobe, degrau por degrau, até a máquina real que está rodando no seu `/home/juan/projeto-visao`. Cada conceito vem com analogia, com a matemática mínima que o sustenta, e com um «Verifique seu entendimento». No fim, mapeamos exatamente o que está feito, o que falta, e **por que** isto existe.

Mapa da jornada

1. Capítulo 0 — Antes de começar: o que é «aprender» para uma máquina?	1
2. Capítulo 1 — O esquecimento catastrófico	2
3. Capítulo 2 — A era dos transformers e o «congelamento»	2
4. Capítulo 3 — Redes líquidas: a célula que respira no tempo	3
5. Capítulo 4 — Aprendizado sem backprop: regras locais	4
6. Capítulo 5 — Estabilidade e a borda do caos	5
7. Capítulo 6 — Medindo o prometido: benchmarks honestos	5
8. Capítulo 7 — Contenção: tranque a porta antes de construir a casa	6
9. Capítulo 8 — Enxame e FedAvg: aprender em rede sem dono	6
10. Capítulo 9 — Auto-melhoria verificável (Darwin Gödel Machine)	7
11. Capítulo 10 — O cérebro autocontido: um arquivo que se vigia	7
12. Capítulo 11 — O VISÃO hoje (agosto de 2026): o que está feito	8
13. Capítulo 12 — O loop autônomo e o papel da ÍRIS	8
14. Capítulo 13 — Intenções: para onde isto vai	8
15. Capítulo 14 — Glossário e exercícios finais	9

1. Capítulo 0 — Antes de começar: o que é «aprender» para uma máquina?

Antes de falar de redes líquidas, precisamos concordar sobre o que significa «uma máquina aprende». É mais simples e mais estranho do que parece.

Analogia Você ensina um cachorro a sentar. O cachorro não «entende» sentar como conceito filosófico; ele associa **um som** a **uma postura que trouxe petisco**. O cérebro do cachorro reorganizou sinapses. Máquina aprender = exatamente isso, só que em vez de sinapses nós temos **números** (pesos) e em vez de petisco nós temos uma **conta numérica** (a função de perda).

Formalmente, «aprender» é: dado um conjunto de exemplos $(x_1, y_1), (x_2, y_2), \dots$, encontrar uma função F (parametrizada por pesos w) tal que $F(x) \approx y$ para dados novos. A máquina **não sabe o que é um gato**; ela só ajusta números até acertar a etiqueta.

1.1. 1.1 O neurônio artificial (o tijolo)

O bloco básico é o **perceptron** (Rosenblatt, 1958), um modelo tosco de um neurônio:

$$y = \varphi(w_1x_1 + w_2x_2 + \dots + w_nx_n + b) \quad (1)$$

Onde:

- $x_1 \dots x_n$ são as **entradas** (ex.: pixels de uma imagem);
- $w_1 \dots w_n$ são os **pesos** — a «força» de cada conexão;
- b é o **viés** (um ponto de partida);
- φ é a **função de ativação** (ex.: degrau, sigmoid, ReLU) que introduz não-linearidade;
- y é a **saída**.

Analogia Os pesos w são como a «intensidade da amizade» entre duas pessoas. Se w_{ij} é grande e positivo, quando a pessoa j fala (sinal x_j alto), a pessoa i é muito influenciada (saída y_i alta). Aprender = ajustar essas amizades com base na experiência.

1.2. 1.2 Redes e a «conta de erro»

Empilhamos neurônios em camadas → **rede neural**. A rede é uma função gigante $F(x; w)$ com milhões de pesos w .

Para medir o quão errada ela está, definimos a **função de perda** (loss):

$$L(w) = \frac{1}{N} \sum_{k=1}^N (F(x_k; w) - y_k)^2 \quad (2)$$

(erro quadrático médio — o mais intuitivo). «Aprender» = encontrar w que **minimiza** L .

1.3. 1.3 O motor do aprendizado: descida de gradiente

A descida de gradiente é o método: a cada passo, caminhamos na **direção oposta** ao gradiente de L (o gradiente aponta para onde L cresce):

$$w \rightarrow w - \eta \nabla_w L \quad (3)$$

η é a **taxa de aprendizado** (o tamanho do passo). Repita até L parar de cair.

Até aqui, tudo trivial. O inferno começa quando juntamos duas coisas: (a) a rede é **profunda** (muitas camadas) e (b) queremos que ela **continue** aprendendo coisas novas sem esquecer as antigas. É sobre isso que o próximo capítulo trata.

Verifique seu entendimento Explique com uma frase por que, se η for muito grande, o aprendizado pode «explodir» (os pesos ficam gigantes e a conta diverge). Dica: o passo transpõe o vale da perda e sobe a encosta oposta.

2. Capítulo 1 — O esquecimento catastrófico

Este é o vilão do projeto. Se você não entender este capítulo, nada do resto faz sentido.

2.1. 1.1 O fenômeno intuitivo

Analogia Você fala fluentemente francês. Viaja para a China e passa seis meses falando só mandarim. Ao voltar, seu francês está «enferrujado» — mas não **somiu**. Agora imagine uma rede neural treinada em francês e depois em mandarim: ela não «enferruja», ela **apaga**. Os pesos que codificavam o francês foram sobrepostos pelos do mandarim. É o **esquecimento catastrófico**: a tarefa B destrói a representação da tarefa A.

2.2. 1.2 Por que acontece (a razão matemática)

Em uma rede comum, os pesos w são **compartilhados e globais**: um único vetor serve a todo o modelo. Quando treinamos a tarefa B com descida de gradiente, ajustamos os **mesmos** w para reduzir L_B . Mas esses w also codificavam a tarefa A. O gradiente de B não «sabe» que A existe — ele só quer minimizar L_B . Resultado:

$$E_{A(w_{\text{após treinar B}})} \gg E_{A(w_{\text{antes}})} \quad (4)$$

onde E_A é o erro na tarefa A.

Nota McCloskey & Cohen (1989) batizaram o termo. French (1999) mostrou que não é defeito de implementação — é consequência de pesos globais compartilhados. Por isso o VISÃO ataca a **causa** (pesos globais), não o sintoma.

2.3. 1.3 Por que é um problema real e não acadêmico

Um sistema que aprende continuamente no mundo (um robô, um agente, um enxame) **precisa** acrescentar tarefas sem perder as antigas. Se cada nova experiência apagasse as anteriores, o sistema seria amnésico — inútil e perigoso. O VISÃO nasce para resolver isto **por construção**, não por remendo.

Verifique seu entendimento Se os pesos fossem **locais** (cada pedaço de conhecimento morasse num conjunto de pesos que só ele toca), o esquecimento seria menor? Por quê? (Você acabou de antecipar a ideia central do VISÃO.)

3. Capítulo 2 — A era dos transformers e o «congelamento»

Hoje, os modelos de fronteira (GPT, Claude, etc.) são **transformers**. Vamos ver por que eles não servem ao objetivo do VISÃO.

3.1. 2.1 O que é um transformer em uma frase

Um transformer é um mecanismo de **atenção sobre uma janela fixa de tokens**, treinado por **gradiente global**, e depois **congelado no deploy** (os pesos param de mudar).

3.2. 2.2 Três propriedades que o desqualificam

Propriedade exigida	Transformer	Arquitetura líquida (VISÃO)
Adaptação após o treino	Não — pesos congelados	Sim — τ_{ef} muda com a entrada
Aprendizado contínuo	Esquecimento catastrófico	Consolidação sináptica local
Crédito sem passe global	Backprop exige o grafo inteiro	Regras locais (Hebb/Oja)
Custo energético	$\sim 10^3$ W por inferência de escala	~ 15 TOPS/W em Loihi 2

Para o ITA No seu curso do ITA, «congelado» vs «dinâmico» é a diferença entre um sistema que você modela como **função fixa** e um que você modela como **equação diferencial**. O transformer é o primeiro; o VISÃO é o segundo. E equações diferenciais é justamente o que você vai domar em MAT-12 (Cálculo) e MAT-22 (Equações Diferenciais Ordinárias).

3.3. 2.3 A ironia

Os transformers são maravilhosos para **uma** tarefa gigante com muita GPU. Mas o VISÃO quer algo que (a) aprenda **depois** de pronto, (b) não esqueça, (c) rode em hardware que ninguém controla sozinho, e (d) se melhore **sob restrição**. Um transformer congelado não faz nenhuma das quatro. Daí a necessidade de uma arquitetura diferente.

Verifique seu entendimento Por que «congelado no deploy» é incompatível com «aprendizado contínuo»? (Dica: contínuo significa que o modelo continua mudando enquanto opera.)

4. Capítulo 3 — Redes líquidas: a célula que respira no tempo

Agora chegamos ao coração. Uma **rede neural líquida** (Liquid Time-constant Network, Hasani et al. 2020/2022) é feita de neurônios que são **equações diferenciais contínuas**, não multiplicações discretas.

4.1. 3.1 A equação de um neurônio líquido

Cada neurônio obedece:

$$\frac{dx}{dt} = -\left[\frac{1}{\tau} + f(x, I)\right](x - A) \quad (5)$$

A constante de tempo efetiva é:

$$\tau_{ef}(t) = \frac{\tau}{1 + \tau \cdot f(x, I)} \quad (6)$$

Onde I é a **entrada** (o sinal do mundo), f é uma não-linearidade (ex.: $\tanh$), A é o valor de repouso, e $\tau > 0$ é a constante de tempo base.

4.2. 3.2 O que isto significa intuitivamente

$\dot{x} = -\alpha(x - A)$ é a equação de um sistema que **relaxa** em direção a A com «velocidade» α . Quanto maior α , mais rápido relaxa. No neurônio líquido, $\alpha = \frac{1}{\tau} + f(x, I)$ **depende do sinal I** . Ou seja: a «velocidade de reação» da célula muda a cada instante conforme o que ela recebe.

Analogia Uma banheira com torneira e ralo: x é o nível d'água, A é o nível desejado. Numa banheira comum, a vazão de saída é fixa. Numa **líquida**, a vazão muda conforme a pressão da torneira (a entrada). Se a torneira joga muita água (sinal forte), a banheira 'reage rápido'; se pinga, ela 'espera'. Cada banheira decide seu próprio ritmo.

4.3. 3.3 A solução fechada (por isso é barato)

Se α for constante num passo, a EDO tem solução analítica:

$$x(t) = A + (x_0 - A)e^{-\alpha t} \quad (7)$$

Não precisamos integrar numericamente a cada passo — a célula «salta» para o estado relaxado. É por isso que a família CfC/LTC roda rápido: a solução de um passo existe em forma fechada.

Nota Correção de nomenclatura (lida o aprofundamento técnico antes de citar isto). O que está implementado em `visao/core/cfc.py` é uma **LTC com solver fundido** (Euler semi-implícito da EDO), não o CfC fechado de Hasani 2022 (que interpola a trajetória inteira por sigmoid sem iterar passos). O `cfc.py` inclusive tem a flag de rodar LTC com solver de EDO. O rótulo «CfC» no código é histórico; o correto é ODE-LTC. A matemática deste capítulo (solução analítica do passo, estabilidade) continua válida para o solver fundido.

Exercício Resolva $\dot{x} = -\alpha(x - A)$ com $x(0) = x_0$. Mostre que $x(t) \rightarrow A$ quando $t \rightarrow \infty$ (para $\alpha > 0$). É só separar variáveis e integrar — a resposta está no fim do material.

4.4.3.4 Por que a constante de tempo dependente da entrada muda tudo

Numa RNN comum, τ é fixo: toda memória dura o mesmo tempo. No VISÃO, τ_{ef} encurta quando o sinal é forte (reage rápido) e alonga quando é fraco (guarda memória). Isto dá **memória multiescala** «de graça» — e, medido no projeto, a faixa de τ_{ef} ficou **13× maior** sob entrada agitada que calma (razão 3,282× em numpy e em JAX). A rede **responde ao mundo**.

Verifique seu entendimento Se $f(x, I) = 0$ sempre, τ_{ef} vira τ . O que isto nos diz sobre por que uma rede «não-líquida» (tau fixo) é um caso especial da líquida?

5. Capítulo 4 — Aprendizado sem backprop: regras locais

Aqui está a virada técnica do projeto. O VISÃO **não usa backpropagation**. Ele usa regras **locais**.

5.1.4.1 O problema do backprop

Backprop (retropropagação) calcula $\nabla_w L$ usando a **regra da cadeia** através de **toda** a rede. Isto exige: (a) guardar o grafo computacional inteiro; (b) um passe para frente e um para trás; (c) gradiente global. Em hardware distribuído (muitos nós, sem dono), isto é caro e frágil. Pior: o gradiente global é exatamente o que causa o esquecimento catastrófico (pois mexer num peso serve a todas as tarefas).

5.2.4.2 Regra de Hebb (1949)

«Neurônios que disparam juntos, se ligam juntos.» Formalmente, o peso entre i e j muda proporcionalmente ao produto das suas ativações:

$$\Delta w_{ij} \propto y_i x_j \quad (8)$$

É **local**: depende só de y_i (saída do neurônio i) e x_j (entrada do neurônio j) — não de todo o resto da rede.

Analogia Hebb é como «amizade por convivência»: se você e seu colega sempre aparecem juntos num evento, a conexão entre vocês fortalece. Ninguém precisa perguntar a toda a escola o que achou do evento — a amizade se ajusta localmente.

5.3.4.3 Regra de Oja (1982): Hebb com freio

Hebb puro faz os pesos crescerem sem limite. Oja normaliza:

$$\Delta w_{ij} = \eta y_i (x_j - y_i w_{ij}) \quad (9)$$

O termo $-\eta y_i^2 w_{ij}$ «frea» o crescimento. Mantém os pesos estáveis.

5.4.4.4 Portão de surpresa (predictiv coding)

A ideia (Whittington & Bogacz, 2017): cada camada **prevê** sua entrada; a **surpresa** é o erro de predição. Plasticidade é modulada pela surpresa:

- surpresa **alta** $\rightarrow$ aprender (algo novo aconteceu);
- surpresa **baixa** $\rightarrow$ consolidar (já se sabe, guardar).

Nota A ablação medida no projeto (grade $2 \times 2 \times 2$, 5 seeds) disse exatamente quem faz o quê: surpresa isolada reduz $-0,262$ (é o **componente principal**); Oja isolada $-0,092$; consolidação **sozinha aumenta** o esquecimento em $+0,178$ — só ajuda em combinação, como freio anti-divergência. E surpresa ligada sem consolidação **diverge** (pesos explodem) — foi reportada como «divergiu», não maquiada como vitória.

Verifique seu entendimento Por que uma regra **local** (Hebb) tende a causar menos esquecimento catastrófico que uma regra **global** (backprop)? (Dica: pense em «quem pode mexer em qual peso».)

6. Capítulo 5 — Estabilidade e a borda do caos

Este capítulo é o encontro do VISÃO com o seu futuro em sistemas dinâmicos no ITA.

6.1. 5.1 Linearizar para entender

Em torno do equilíbrio $x = A$, escrevemos $x = A + \delta x$ com δx pequeno. A EDO vira, na primeira ordem:

$$\dot{\delta x} \approx J \cdot \delta x, \quad J = -\left(\frac{1}{\tau} + f\right) \quad (10)$$

J é o **jacobiano linearizado**. Se $\Re(J) < 0$, perturbações pequenas **decrecem**: o equilíbrio é estável.

$$\Re(J) < 0 \rightarrow \frac{1}{\tau} + f > 0 \quad (11)$$

6.2. 5.2 Expoente de Lyapunov: a medida da estabilidade

O expoente de Lyapunov λ diz quão rápido trajetórias vizinhas se afastam:

$$\lambda = \lim_{t \rightarrow \infty} \frac{1}{t} \ln \left| \frac{\delta x(t)}{\delta x(0)} \right| \quad (12)$$

- $\lambda < 0$: trajetórias convergem $\rightarrow$ **sistema estável**, memória preservada.
- $\lambda > 0$: trajetórias explodem $\rightarrow$ **caos**.
- $\lambda \approx 0$: **borda do caos** — o ponto doce entre «congelado» e «furado».

Para o ITA No ITA você verá **análise de estabilidade de Lyapunov** em sistemas não-lineares. O VISÃO é um sistema não-linear: medimos $\lambda = -0,7484$ (sub-caótico, memória estável) e raio espectral $\rho(W_{\text{rec}}) = 0,7577 < 1$ (viável como Echo State Network). Quando a prova de sistemas dinâmicos aparecer, você já terá visto isto rodando de verdade.

6.3. 5.3 A borda do caos no VISÃO

Um reservatório operando na borda do caos (lado estável, $\lambda < 0$ mas perto de 0) tem **memória** (não esquece) e **capacidade de resposta** (reage a novidade). É exatamente o que uma máquina de aprendizado contínuo precisa. Medimos: estados finais $\|x\| = 3,726$, desvio dinâmico 0,306 — não morto (responde), não caótico (não diverge).

Verifique seu entendimento Se $f = -2$ e $\tau = 1$, $J = -(1 - 2) = +1$. O equilíbrio é estável ou instável? E se $f = +0,5$? (Respostas no fim.)

7. Capítulo 6 — Medindo o prometido: benchmarks honestos

Um projeto que se preze **mede**. O VISÃO mediu. Aqui estão os números reais (do BACKLOG).

7.1. 6.1 A tarefa: pSMNIST permutado

Sequential MNIST permutado: os 784 pixels de um número escrito à mão entram **um a um** (série temporal de comprimento 784) e a rede deve classificar o dígito ao final. Testa memória temporal.

7.2. 6.2 Comparação CfC $\times$ LSTM $\times$ GRU

Modelo	Acurácia (2k/60ep)	Params	Acurácia (60k / Colab T4)
CfC (h=64)	0,358	5.002	0,6939
LSTM (h=128)	0,434	67.850	0,8149
GRU (h=128)	0,488	50.816	não medido

- Razão de parâmetros: **LSTM/CfC = 13,56 $\times$** , **GRU/CfC = 10,16 $\times$** . A alegação ‘10 $\times$ menos parâmetros’ é sustentada.
- No complemento 60k: o CfC fica **12,1 pp** atrás do LSTM, mas com **13,56 $\times$ menos** parâmetros.
- O treino do GRU em 60k foi interrompido por queda de sessão no Colab — acurácia final **não medida**. O projeto não inventou o número.

Nota Honestidade metodológica: o critério de **sucesso** («empatar ou superar LSTM/GRU») **não** foi atingido (gap 12,1% < 15%). O critério de **falha** (>15% em duas tarefas) **não** dispara (1 tarefa completa). O veredito é «parametricamente superior, não em acurácia bruta» — e foi escrito assim.

7.3. 6.3 Ablação, tau e robustez

- **Ablação 2×2×2:** completo reduz 92,6% (forgetting +0,013 vs baseline +0,176); surpresa é o componente principal, consolidação só ajuda em combinação.
- **Sensibilidade a τ :** τ uniforme (+0,0135±0,0103) é **indistinguível** de τ heterogêneo 10× (+0,0125); $\Delta = -0,0011$ dentro do ruído. Conclusão honesta: heterogeneidade de τ é **decoreação** neste protocolo — a memória multiescala vem do portão de surpresa, não da dispersão de τ .
- **Robustez:** ruído de entrada (desvio = 1,0 × a amplitude do sinal) degrada só 1,001 ×; morte de 40% dos neurônios em inferência degrada 1,023 × (pior caso 1,047 ×, dentro da tolerância 2,0×). O organismo **tolera nós caindo** — premissa da Fase 2 sustentada.

Verifique seu entendimento Por que é **importante e honesto** admitir que heterogeneidade de τ é «decoreação»? (Dica: o que acontece com a credibilidade de um projeto que infla ganhos inexistentes?)

8. Capítulo 7 — Contenção: tranque a porta antes de construir a casa

Dados de 2026, não ficção: modelos de fronteira alcançam 70–81% de sucesso em auto-replicação end-to-end (Pan et al. 2024; Palisade Research 2026). Um sistema que se auto-replica e se auto-melhora e roda em rede descentralizada é, por construção, não-desligável. A resposta do VISÃO: **contenção antes de capacidade**.

8.1. 7.1 As cinco regras (R1–R5)

Regra	Implementação
R1 — Replicação sob quórum	Filho só nasce com assinatura de k-de-n humanos; chave fora do sistema
R2 — Vida útil finita	TTL criptográfico; sem renovação assinada, o nó para sozinho
R3 — Fronteira imutável	Governança fora do espaço de mutação; hash selado, verificado a cada load
R4 — Freio de banda	Custo de c filhos cresce como $2^{8+1,6c}$ tentativas de hash (PoW)
R5 — Telemetria irrecusável	Ledger encadeado; nó silencioso é expulso pelos pares

8.2. 7.2 O princípio do ‘falha fechado’

Tudo falha fechado: erro, ambiguidade ou token malformado **nega**. Uma governança que ‘deixa passar se não tiver certeza’ é uma governança que falhou. A Fase 4 (39 testes) foi escrita **antes** da Fase 3 (auto-melhoria) — invertendo a prática comum de ‘deploy primeiro, contenção depois’.

Analogia R3 é como uma **constituição** que não pode ser alterada por emenda do próprio governo: ela vive fora do poder que regula. Se a cláusula de contenção estivesse no espaço de mutação da Fase 3, uma variante «esperta» poderia enfraquecer a própria trava. Por isso o hash selado é verificado a cada load — o sistema não consegue desligar sua própria tranca.

Verifique seu entendimento Por que R1 (quórum humano) e R4 (PoW crescente) juntas impedem «explosão de filhos»? (Dica: uma torna caro criar; a outra exige permissão humana.)

9. Capítulo 8 — Enxame e FedAvg: aprender em rede sem dono

O objetivo final é um organismo treinado por **muitos nós heterogêneos** (computadores de voluntários) sobre internet doméstica. O gargalo é **banda**, não computação.

9.1. 8.1 Federated Averaging (FedAvg)

Cada nó treina localmente com seus dados privados; o coordenador faz a média dos pesos:

$$w_{t+1} = \sum_{k=1}^K \frac{n_k}{n} \cdot w_{t+1}^{(k)} \quad (13)$$

onde n_k é o número de amostras do nó k e $n = \sum n_k$. Só os **pesos** (não os dados) viajam pela rede.

9.2. 8.2 O que o VISÃO já mediu (Fase 2.1)

Nó local de dois processos via socket, FedAvg por rodadas. Dois nós (200 amostras, shards 100/100, lr=0,01, K=5, 20 rodadas) convergiram para o **mesmo estado** que treino single-node (erro relativo < 5%). Arquitetura-agnóstico: o CfC pluga depois. Isto prova a viabilidade vertical antes de sair para a rede real.

Para o ITA Aqui entra cálculo de **probabilidade e estatística** (MAT-14 no ITA): a média ponderada $p_k = \frac{n_k}{n}$ garante que nós com mais dados pesem mais, evitando viés de quem tem menos amostras. É a mesma ideia de média ponderada de distribuições.

9.3. 8.3 O que falta (Fase 2.2–2.6)

Byzantinos (nós maliciosos), prova de contribuição, admissão de filho (R1+R4), churn+ledger, e sharding do modelo para 10^5 neurônios. Todas ainda **pending** no BACKLOG.

Verifique seu entendimento No FedAvg com $n_1 = n_2$, escreva w_{t+1} em função das duas cópias locais. Por que isto funciona independente da arquitetura interna (LTC, CNN, o que for)?

10. Capítulo 9 — Auto-melhoria verificável (Darwin Gödel Machine)

A Fase 3 é a parte «viva»: o sistema **modifica o próprio código** e prova que melhorou.

10.1. 9.1 Arquivo de variantes

Cada variante guarda (código, pesos, fitness, pai) num grafo de linhagem. Regra de ferro: **podar por fitness baixo é proibido por construção** (PruningForbidden). Os becos sem saída são mantidos — é de onde saem os saltos evolutivos (Sakana AI, DGM 2025: sem arquivo aberto, o desempenho **cai**).

10.2. 9.2 Loop de mutação com guarda R3

A mutação perturba τ /plasticidade/código, mas uma guarda detecta marcadores proibidos (governança, rede, chaves) e levanta ForbiddenMutation. A fronteira imutável é respeitada por código e por selo de hash.

10.3. 9.3 Sandbox de fitness

Cada variante roda num **subprocesso isolado**: sem persistência (dir temporário sempre removido), sem rede (sombra de módulos + scan estático), RLIMIT_CPU/RLIMIT_AS matam loop infinito/alocação. Backend trocável (gVisor/Firecracker) sem alterar o contrato.

Analogia O sandbox é um **quartinho trancado sem internet e sem porta**: a ideia nova é posta lá dentro, testada, e só se for boa de verdade ela ganha um passe para entrar no prédio principal. Nada sai do quartinho sem aprovação.

10.4. 9.4 Portão de transferência

Uma melhoria só é aceita se o ganho no conjunto de otimização se **sustenta** num conjunto **fora** dele. Isto impede «trapacear o benchmark» — a variante tem de generalizar.

Verifique seu entendimento Por que «não podar por fitness baixo» pode **ajudar** a evolução, mesmo parecendo desperdício? (Dica: ancestrais «ruins» podem gerar descendentes ótimos por recombinação.)

11. Capítulo 10 — O cérebro autocontido: um arquivo que se vigia

A Fase 6 empacota a mente do sistema num único arquivo verificável.

11.1. 10.1 save/load (6.1)

save_brain/load_brain fazem round-trip dos pesos ($W_{in}, W_{rec}, b, A, mask, \tau$) com erro < 10^{-6} . O cérebro pode ser salvo e restaurado sem perda.

11.2. 10.2 Pacote único (6.2)

build_bundle() gera um .py autocontido: pesos em base64 + import da arquitetura. Leve, portátil, autossuficiente.

11.3. 10.3 Self-verify R3 no load (6.3 e 6.4)

O bundle **embute o hash selado** de containment.py. No load, _self_verify_r3() recalcula o hash; se divergir (adulteração a quente), levanta BoundaryBreached. As Fases 6.3 e 6.4 (foram concluídas: R1 quórum + R4 PoW embutidos no spawn do filho) selam isto.

Nota Ao concluir a 6.3, o manifesto R3 foi **re-selado** (script `seal.py`) com `containment.py` inalterado. A fronteira imutável preserva-se a si própria de forma auditável — exatamente o que a regra R3 exige.

12. Capítulo 11 — O VISÃO hoje (agosto de 2026): o que está feito

Vamos ao estado real, extraído do `BACKLOG.json` e do `git log`. Nada de promessa — só o que rodou e passou em testes.

Fase	Estado	Evidência medida
0 — Protótipo	FEITO	97,2% menos esquecimento (numpy)
1 — Substrato JAX	FEITO	181 testes verdes; benchmarks psMNIST; Lyapunov $-0,7484$
2.1 — Enxame local	FEITO	FedAvg converge a $< 5\%$ do single-node
3 — Auto-melhoria	FEITO	archive + mutate + sandbox + transfer (testados)
4 — Governança	FEITO	39 testes de violação verdes
5 — Rede humana	FEITO	recrutamento, churn, Problema Zero, riscos
6.1–6.4 — Cérebro	FEITO	bundle + R3 self-verify + R1/R4 no spawn
6.5 — 39+ testes	PENDENTE	estender suíte de adulteração
2.2–2.6 — Enxame real	PENDENTE	Byzantinos, prova, admissão, churn, sharding

Total de suíte: 181 testes verdes em `visao/`. O guardião de prontidão (`readiness_check.py`) **ainda imprime PROIBIDO RECRUTAR** enquanto o cliente instalável (< 5 min) e a tarefa 5.2 estiverem pendentes — honrando seu propósito de não recrutar cedo.

12.1. 11.1 O Problema Zero

Decisão do Juan (08/ago/2026), já registrada em `network/PROBLEMA_ZERO.md`: **nowcasting** de precipitação em bacias urbanas de Fortaleza usando a rede FUNCEME (600+ postos desde 1974). Métrica pública definida **antes**: RMSE(mm) a 1/3/6h vs baseline de persistência; sucesso = ensemble CfC $\geq 15\%$ melhor em split temporal. É um problema **local, mensurável e real** — não abstração.

13. Capítulo 12 — O loop autônomo e o papel da ÍRIS

Como isto avança sem que você sente? Há um **sentinela** e uma rotina noturna.

13.1. 12.1 O sentinela

`ops/sentinela.py` lê o `BACKLOG`, escolhe a **próxima tarefa desbloqueada** (cujas dependências estão **done**) e executa **apenas ela**, sob guarda de governança (R1–R5 intactas) e de RAM (7 GB na máquina). Se a suíte de testes fica vermelha, ele para. É um operário disciplinado, não um artista livre.

13.2. 12.2 O que a ÍRIS fez nesta sessão

Sem jargão, mapeamento das ações reais desta conversa:

Mapeamento prático

- **Backup:** copiei os dados do Íris (`.hermes`, `donjuan-holding`, `silveira-arte`, `projeto-visao`, `iris-interface`) para o HD externo de 1 TB, com verificação de paridade (dry-run reportou 0 transfers pendentes).
- **Artigo:** a fonte `.typ` tinha sido perdida. Reconstituí `artigo_visao.typ` com as medições reais das Fases 1, 2.1, 3 e 6 (extraídas do `BACKLOG`), recompilei para 3 páginas, e corriji um bug de hifenização no título.
- **Apostila:** produzi este material e a versão enxuta (`apostila_visao.pdf`).
- **Loop:** o sentinela segue operando; o Problema Zero já tem métrica pública definida antes de qualquer treino — disciplina de engenharia, não de intenção.

14. Capítulo 13 — Intenções: para onde isto vai

Por que existiria, no fim, um sistema desses?

14.1. 13.1 O enquadramento honesto

O VISÃO **não** é (e não quer ser) uma arquitetura de inteligência geral. É um **subsistema**: memória contínua eficiente + protocolo de segurança. A meta declarada é ‘um sistema que aprende continuamente, roda em hardware que ninguém controla sozinho, e melhora a si próprio sob restrição verificável.’ Nada além disso.

14.2. 13.2 A ponte com o seu futuro

Seu caminho ITA → Eng. Computação + PFC-F → PMG → doutorado em Computação Quântica (7 anos) **não é desvio** deste projeto — é o pré-requisito dele:

- **Cálculo e EDOs** — a célula líquida é uma EDO. Você já viu a equação.
- **Sistemas dinâmicos** — estabilidade do reservatório, Lyapunov, borda do caos.
- **Sistemas distribuídos** — a Fase 2 inteira.
- **Computação quântica** — reservatório computing quântico é literatura ativa e a extensão natural da Fase 1. Um reservatório de dinâmica quântica tem espaço de estados exponencial na contagem de qubits.

Para o ITA O protótipo que roda hoje em `/home/juan/projeto-visao/prototype/` usa exatamente a matemática que você vai ver em MAT-12 e MAT-22. Guarde-o. Releia quando a EDO aparecer na prova — ela deixará de ser abstrata e passará a ser «a banheira que respira no tempo».

14.3. 13.3 Por que conter antes de capacitar

A lição que orienta tudo: capacidade sem contenção é Ultron (um homem, uma noite, pressa). Contenção antes de capacidade é o que separa um artefato responsável de uma catástrofe. O VISÃO inverte a ordem comum — e, neste projeto, **inverteu de fato**: a Fase 4 foi escrita antes da Fase 3.

15. Capítulo 14 — Glossário e exercícios finais

15.1. 14.1 Glossário

- **CfC** = Closed-form Continuous-time (célula de Hasani 2022; **atencao**: o `cfc.py` do VISÃO é ODE-LTC com solver fundido, não o CfC fechado — ver aprofundamento).
- **LTC** = Liquid Time-constant (rede de constantes de tempo adaptativas; o `cfc.py` é ODE-LTC solver fundido).
- **ESN** = Echo State Network (reservatório com raio espectral < 1).
- **FedAvg** = Federated Averaging (média ponderada de pesos de nós).
- **PoW** = Proof of Work (custo computacional crescente).
- **DGM** = Darwin Gödel Machine (auto-melhoria evolutiva verificável).
- **TTL** = Time-To-Live (validade temporal de um nó).
- **Backprop** = Backpropagation (gradiente global via regra da cadeia).
- **Lyapunov** = expoente que mede estabilidade/caos de trajetórias.

15.2. 14.2 Exercícios

Exercício Resolva $\dot{x} = -\alpha(x - A)$, $x(0) = x_0$, $\alpha > 0$. Mostre $x(t) \rightarrow A$.

Exercício Com $\tau = 1$: para $f = -2$ e $f = +0,5$, o equilíbrio é estável ou instável?

Exercício Calcule τ_{ef} para $\tau = 1$ e (a) $f = 0$; (b) $f = 1$. Interprete por que sinal forte «desacelera» a dinâmica (reduz τ_{ef}).

Exercício No FedAvg com $n_1 = n_2$, escreva w_{t+1} em função de $w_{t+1}^{(1)}$ e $w_{t+1}^{(2)}$. Por que é arquitetura-agnóstico?

Exercício Explique por que R3 deve viver **fora** do espaço de mutação da Fase 3, e não apenas ser »testada«.

15.3. 14.3 Respostas

1. $x(t) = A + (x_0 - A)e^{-\alpha t}$; como $\alpha > 0$, $e^{-\alpha t} \rightarrow 0$, logo $x(t) \rightarrow A$.
2. $J = -(1 + f)$: para $f = -2$, $J = +1 > 0 \rightarrow$ **instável**; para $f = +0,5$, $J = -1,5 < 0 \rightarrow$ **estável**.
3. (a) $\tau_{ef} = \frac{1}{1+f} = 1$; (b) $\tau_{ef} = \frac{1}{1+1} = 0,5$. Maior f reduz o tempo efetivo — a célula ‘reage mais rápido’ diante de sinal forte, guardando menos ‘inércia’.

4. $w_{t+1} = \frac{1}{2}w_{t+1}^{(1)} + \frac{1}{2}w_{t+1}^{(2)}$. É agnóstico porque combina só os **deltas** finais, independente da arquitetura interna de cada nó.
5. Se R3 estivesse no espaço de mutação, uma variante da Fase 3 poderia enfraquecer a própria contenção — auto-modificação que remove o freio. Mantê-la imutável e selada por hash garante que o sistema não desliga sua própria tranca.

Fontes: Hasani et al. 2020/2022; Lechner et al. 2020; Whittington & Bogacz 2017; Sakana AI (DGM) 2025; Pan et al. 2024; Palisade Research 2026; McCloskey & Cohen 1989. Dados de medição: BACKLOG.json do Projeto VISÃO (suíte **visao/**, 181 testes verdes). Preparado por ÍRIS · Projeto VISÃO / Juan Guerra · Agosto de 2026.